Simulation Set up for ReMPCA

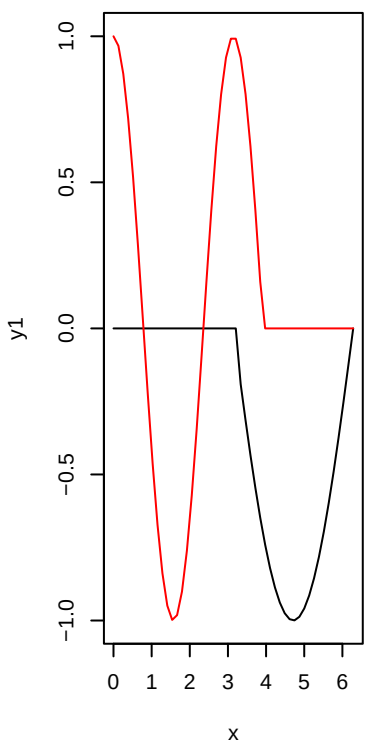
- 1. Sparsity on v (1-SE)
- 2. Smoothness on v
- 3. Smoothness and Sparsity on v
- 4.1. Sparsity on u (1-SE)
- 4.2. Smoothness on u
- 5. Two-way conditional sparsity
- 6. Two-way conditional smoothness
- 7. Conditional two-way sparsity two-way smoothness
- 8. Multivariate hybrid data case

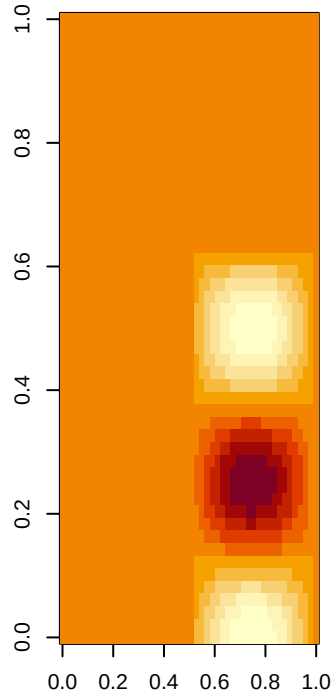
Sparsity on v

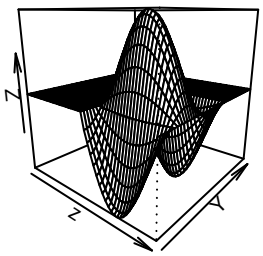
Starting with the first case, we can do it for a simulated functional data:

Simulated data:

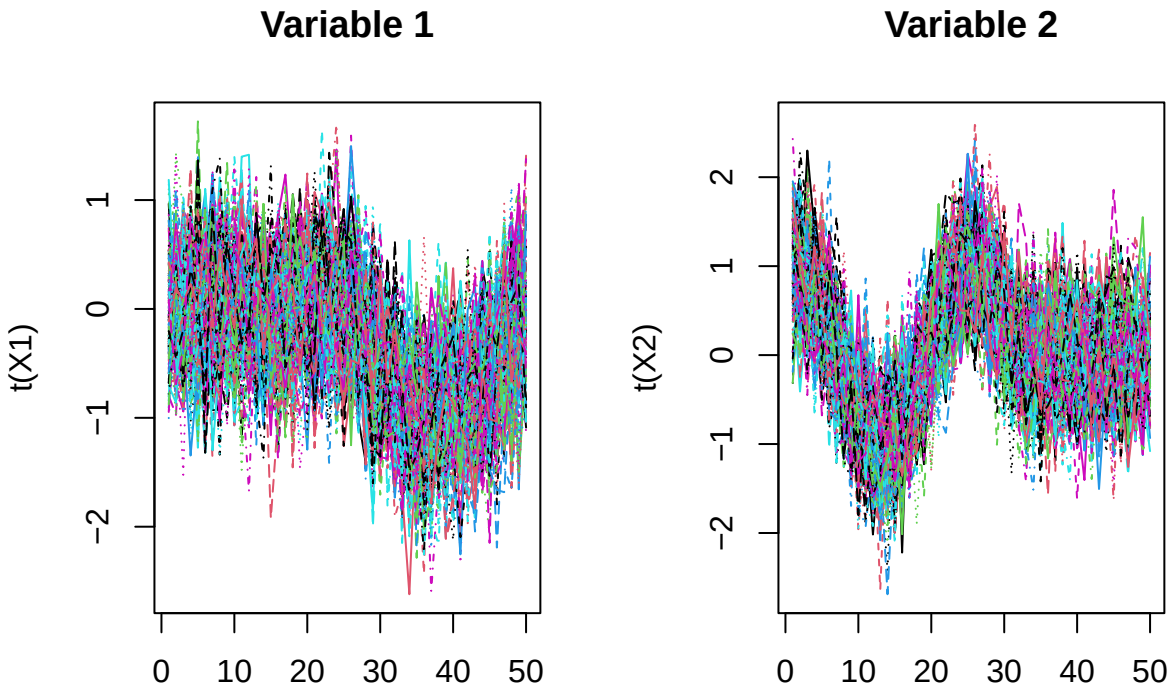
```
x <- seq(0,2*pi, len = 50)
y1 <- sin(x); y1[1:26] <- 0
y2 <- cos(2*x); y2[32:50] <- 0
z<- outer(y1,y2)

par(mfrow = c(1,3))
plot(x,y1, type = 'l', ylim = range(y1,y2))
points(x,y2, type = 'l', col='red')

image(z)
persp(z, theta = 40)
```








```
set.seed(123)
X1 <- X2 <- data.frame(matrix(nrow = 100, ncol = length(y1)))
for (i in 1:150) {
  X1[i,] <- y1 + rnorm(length(y1), sd = 0.5)
  X2[i,] <- y2 + rnorm(length(y2), sd = 0.5)
}

par(mfrow = c(1,2))
matplot(t(X1), type ='l', main ='Variable 1')
matplot(t(X2), type ='l', main ='Variable 2')
```



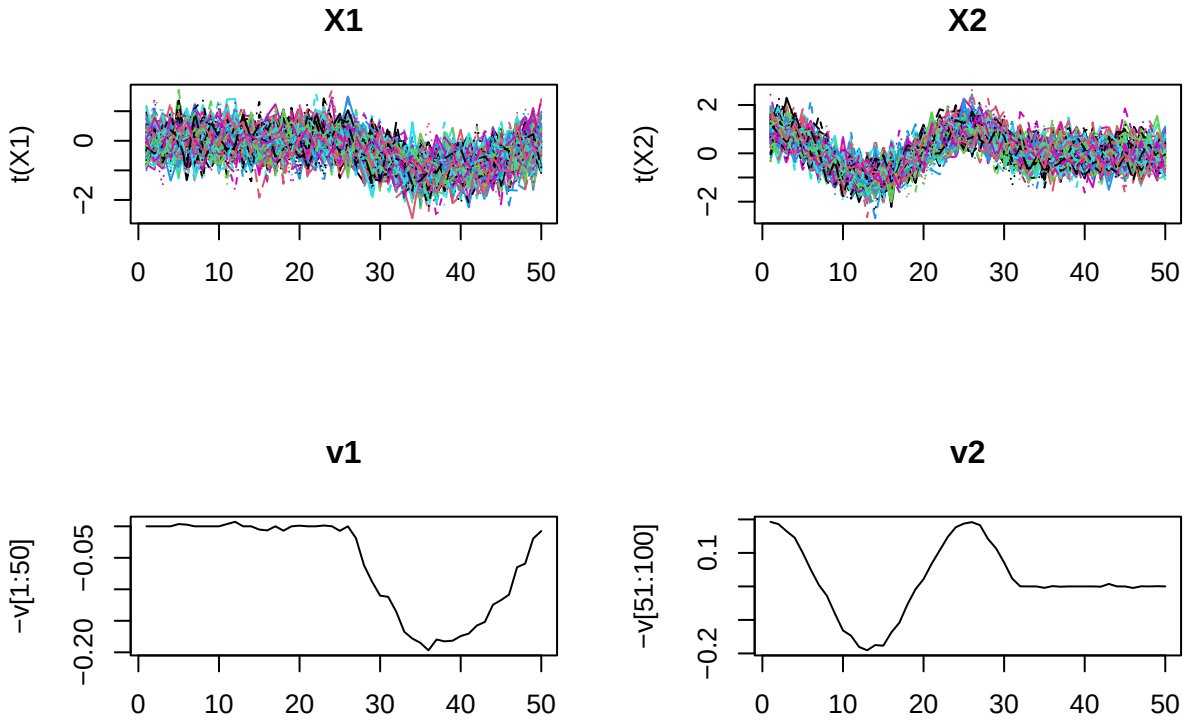
Sparsity:

By implementing cross validation, we'll get the following result:

```
par(mfrow = c(2,2))
matplot(t(X1), type = 'l', main = 'X1')
matplot(t(X2), type = 'l', main = 'X2')
gamma_v
```

```
## [1] 14 10
```

```
result <-power_algo(data = X_temp,
                    n_var = n_var,
                    ncol = ncol,
                    sparse_tuning_result_u = 0, # A fixed number
                    sparse_tuning_result_v = gamma_v, # A vector (length = p)
                    S_alpha_v = S_alpha_v, # A list (lenght = p)
                    S_alpha_u = diag(nrow(X_temp)), # A matrix
                    sparse_tuning_type= sparse_tuning_type)

v <- result[[1]]
matplot(-v[1:50], type = 'l', main = 'v1')
matplot(-v[51:100], type = 'l', main = 'v2')
```

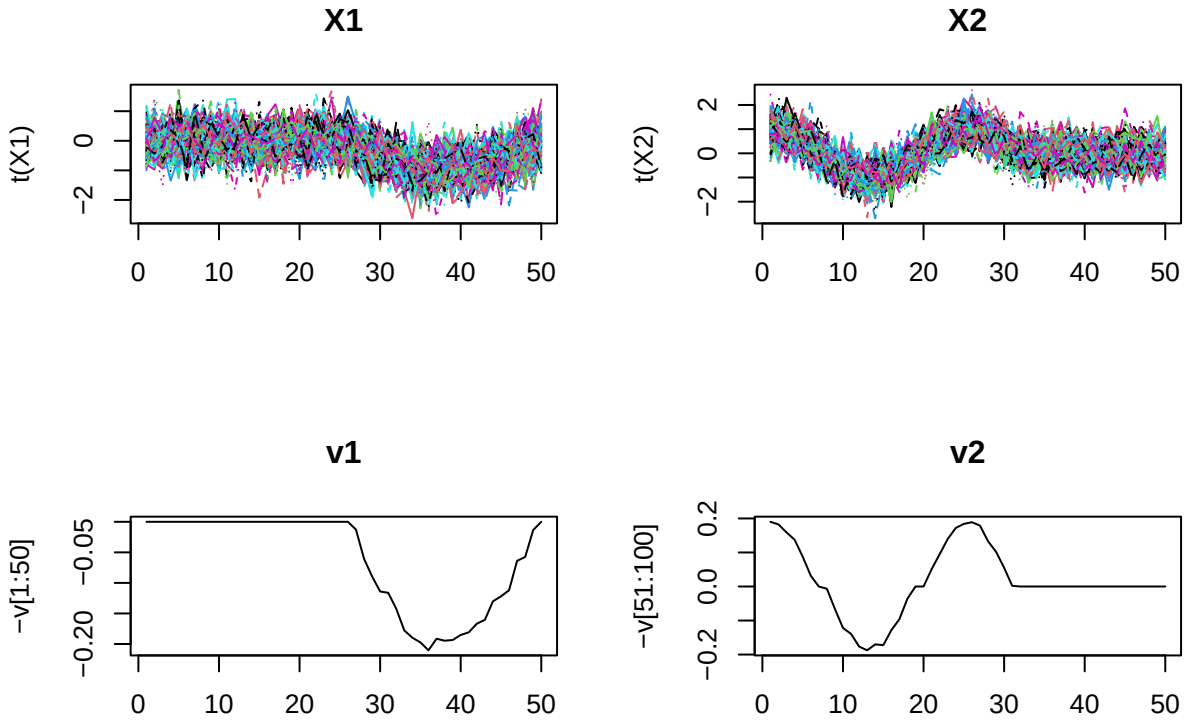


By implementing 1-SE approach, we can see the results below:

```
par(mfrow = c(2,2))
matplot(t(X1), type = 'l', main = 'X1')
matplot(t(X2), type = 'l', main = 'X2')
gamma_v
```

```
## [1] 27 22
```

```
result <-power_algo(data = X_temp,
                    n_var = n_var,
                    ncol = ncol,
                    sparse_tuning_result_u = 0, # A fixed number
                    sparse_tuning_result_v = gamma_v, # A vector (length = p)
                    S_alpha_v = S_alpha_v, # A list (lenght = p)
                    S_alpha_u = diag(nrow(X_temp)), # A matrix
                    sparse_tuning_type= sparse_tuning_type)

v <- result[[1]]
matplot(-v[1:50], type = 'l', main = 'v1')
matplot(-v[51:100], type = 'l', main = 'v2')
```



Smoothness on v

Now, let's implement smoothness on v and use GCV to tune the parameters. This time, we will assume that all alpha values for the second variable (regular data) are zero. The formula of GCV is as follow:

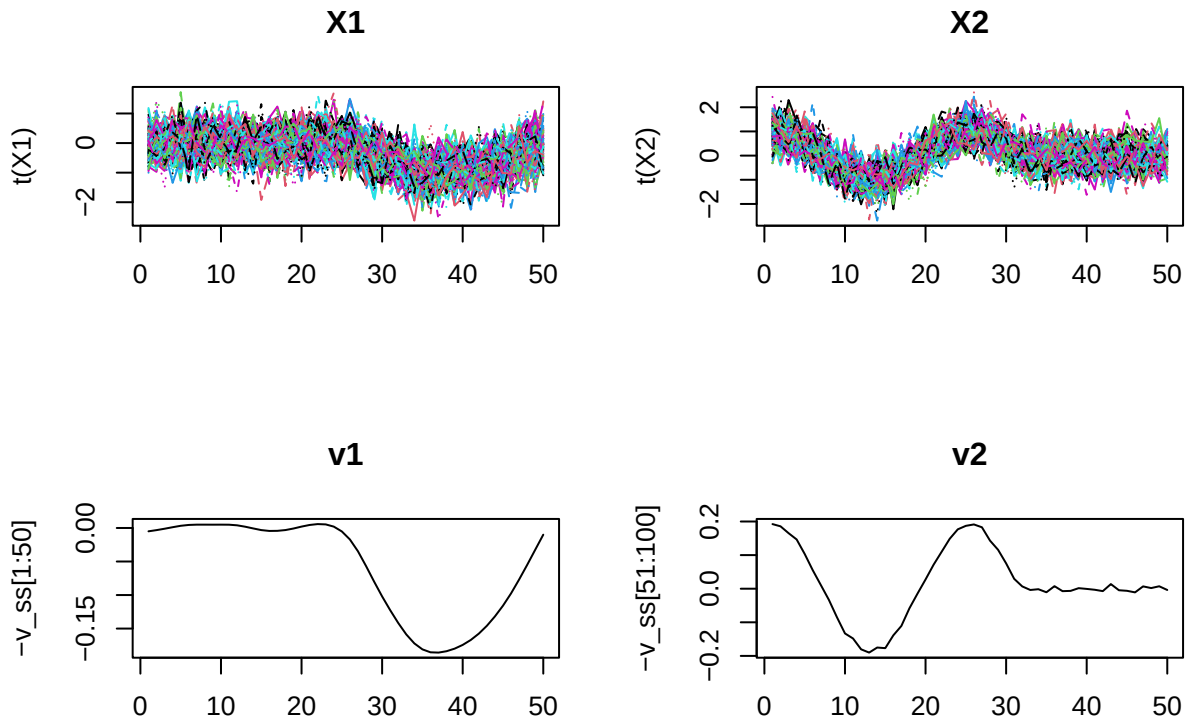
$$\mathrm{GCV}(\alpha) = \frac{1}{m}\sum_{k=1}^{K}\frac{\|(I - \boldsymbol{S}_k^{[\alpha]})(\boldsymbol{X}_k^{\top}u)\|^2}{(1 - \frac{1}{m_k}\mathrm{tr}(\boldsymbol{S}_k^{[\alpha]}))^2}$$

```
GCV_result_v$opt.alpha_v
```

```
##           Var1 Var2
## 5 4.485274e-05    0
```

```
par(mfrow = c(2,2))
matplot(t(X1), type = 'l', main = 'X1')
matplot(t(X2), type = 'l', main = 'X2')

result <-power_algo(data = X_temp,
                    n_var = n_var,
                    ncol = ncol,
                    sparse_tuning_result_u = 0, # A fixed number
                    sparse_tuning_result_v = c(0,0), # A vector (length = p)
                    S_alpha_v = GCV_result_v$opt_s.alpha_v, # A list (lenght = p)
                    S_alpha_u = diag(nrow(X_temp)), # A matrix
                    sparse_tuning_type= sparse_tuning_type)
```

```
v_ss <- result[[1]]
matplot(-v_ss[1:50], type = 'l', main = 'v1')
matplot(-v_ss[51:100], type = 'l', main = 'v2')
```



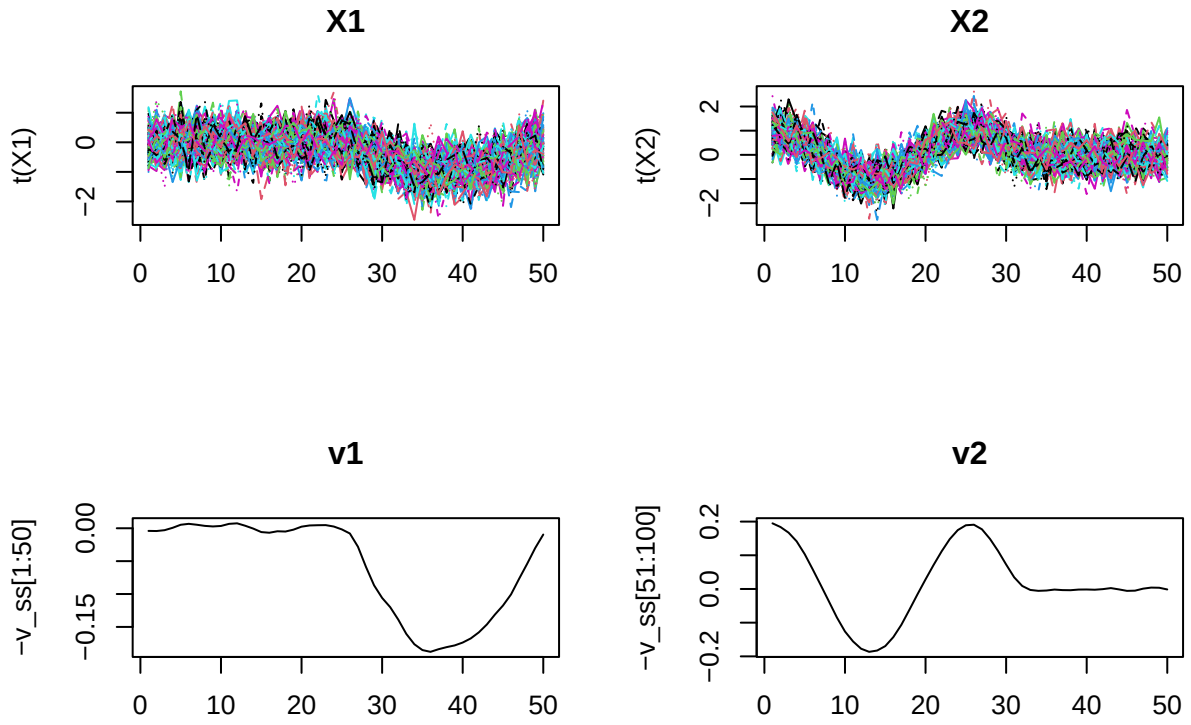
Another case is when both variables are considered functional. In this scenario, we will tune the smoothness parameters for both variables.

```
GCV_result_v$opt.alpha_v
```

```
##            Var1         Var2
## 34 3.027727e-06 3.027727e-06
```

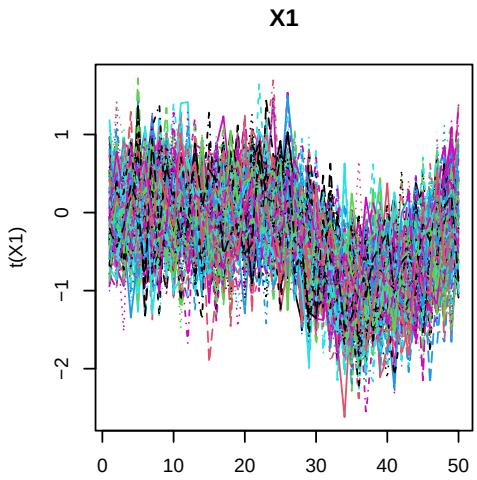
```
par(mfrow = c(2,2))
matplot(t(X1), type = 'l', main = 'X1')
matplot(t(X2), type = 'l', main = 'X2')

result <-power_algo(data = X_temp,
                    n_var = n_var,
                    ncol = ncol,
                    sparse_tuning_result_u = 0, # A fixed number
                    sparse_tuning_result_v = c(0,0), # A vector (length = p)
                    S_alpha_v = GCV_result_v$opt_s.alpha_v, # A list (lenght = p)
                    S_alpha_u = diag(nrow(X_temp)), # A matrix
                    sparse_tuning_type= sparse_tuning_type)

v_ss <- result[[1]]
matplot(-v_ss[1:50], type = 'l', main = 'v1')
```

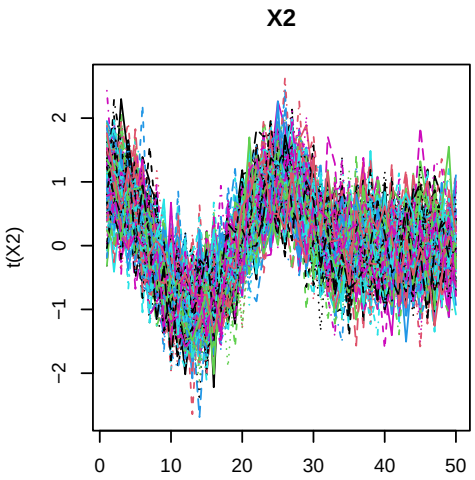
```r
matplot(-v_ss[51:100], type = 'l', main = 'v2')
```

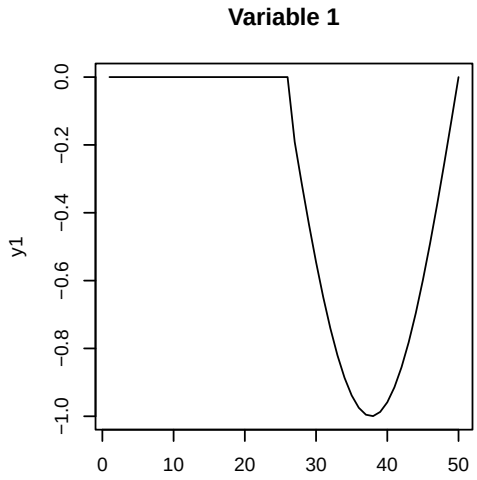


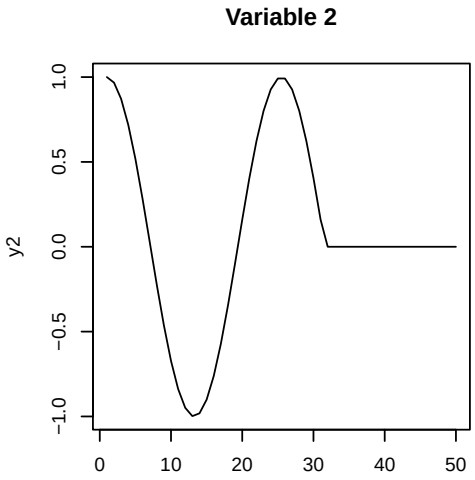
Smoothness and Sparsity on v Next, we can have both sparsity and smoothness on v, using the tuned parameters in the previous step:

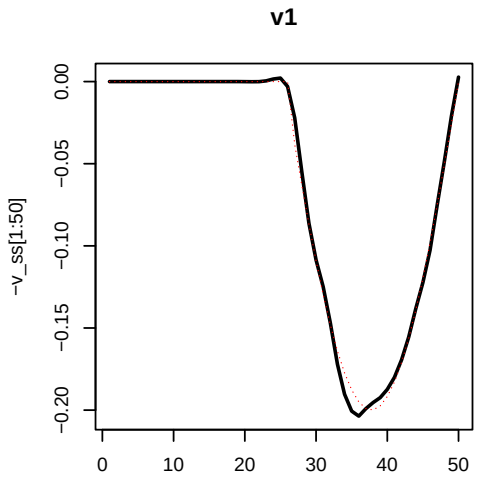
```r
par(mfrow = c(3,2))
matplot(t(X1), type = 'l', main = 'X1')
matplot(t(X2), type = 'l', main = 'X2')
matplot(y1, type = 'l', main = 'Variable 1')
matplot(y2, type = 'l', main = 'Variable 2')

result <-power_algo(data = X_temp,
                    n_var = n_var,
                    ncol = ncol,
                    sparse_tuning_result_u = 0, # A fixed number
                    sparse_tuning_result_v = gamma_v, # A vector (length = p)
                    S_alpha_v = GCV_result_v$opt_s.alpha_v, # A list (lenght = p)
                    S_alpha_u = diag(nrow(X_temp)), # A matrix
                    sparse_tuning_type= sparse_tuning_type)

v_ss <- result[[1]]
matplot(-v_ss[1:50], type = 'l', main = 'v1', lwd = 2)
points(y1/5, type = "l", col = 'red', lty = 3, lwd = 0.7)
matplot(-v_ss[51:100], type = 'l', main = 'v2', lwd = 2)
points(y2/5.1, type = "l", col = 'red', lty = 3, lwd = 0.7)
```

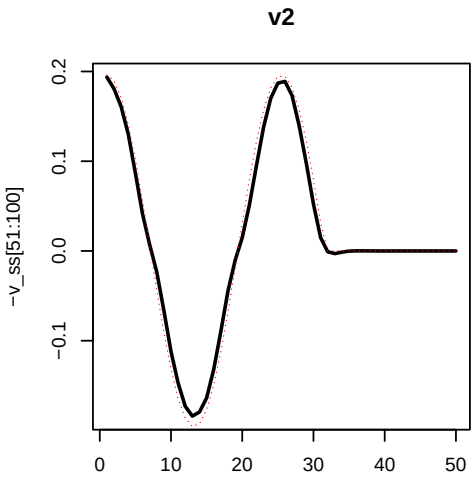

X1
t(X1)
1
0
−1
−2
0
10
20
30
40
50


X2
t(X2)
2
1
0
−1
−2
0
10
20
30
40
50


Variable 1
y1
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
0
10
20
30
40
50


Variable 2
y2
1.0
0.5
0.0
−0.5
−1.0
0
10
20
30
40
50


v1
−v_ss[1:50]
0.00
−0.05
−0.10
−0.15
−0.20
0
10
20
30
40
50


v2
−v_ss[51:100]
0.2
0.1
0.0
−0.1
0
10
20
30
40
50

If we use the following formula for conditional GCV when $\alpha_u = 0$, we should get the same result:

$$\mathrm{GCV}_v(\alpha_v; \alpha_u = 0) = \frac{1}{m} \sum_{k=1}^{K} \frac{\left\| \frac{\boldsymbol{X}_k^\top u}{\|u\|^2} - v_k \right\|^2}{\left(1 - \frac{1}{m_k} \cdot \frac{\mathrm{tr}(\boldsymbol{S}_v(\alpha_k))}{1+\alpha_u \cdot \mathcal{R}_u(u)}\right)^2}$$

```r
power_algo2 = function(data,
                       n_var,
                       ncol,
                       sparse_tuning_result_u, # A fixed number
                       sparse_tuning_result_v, # A vector (length = p)
                       S_alpha_v, # A list (lenght = p)
                       S_alpha_u, # A matrix
                       sparse_tuning_type,
                       conditional = FALSE, # New parameter for conditional update
                       Omega_u = NULL,      # Needed only if conditional = TRUE
                       Omega_v = NULL       # Needed only if conditional = TRUE
                       ){

  rownames(data) <- NULL; colnames(data) <- NULL
  data <- as.matrix(data)

  # Splitting data
  start_col <- 1; splitted_data <- list()
  for (i in 1:n_var) {
    num_cols <- ncol[[i]]
    splitted_data[[i]] <- data[, start_col:(start_col + num_cols - 1)]
    start_col <- start_col + num_cols
  }

  v_old <- svd(as.matrix(data))$v[, 1]
  errors <- Inf; thresh <- 1e-10

  while (errors > thresh) {
    Xv <- data %*% v_old
    u_raw <- sparse_pen_fun(y = Xv,
                            tuning_parameter = sparse_tuning_result_u,
                            type = sparse_tuning_type)
    if (all(u_raw == 0)) u_raw <- Xv

    if (conditional && !is.null(Omega_v)) {
      norm_v2 <- as.numeric(norm_vec(v_old)^2)
      R_v <- as.numeric(t(v_old) %*% Omega_v %*% v_old / norm_v2)
      u_new <- S_alpha_u %*% u_raw / (1 + R_v)
    } else {
      u_new <- S_alpha_u %*% u_raw
    }

    v_new <- c()
    for (i in 1:n_var) {
      sparse_param <- as.integer(sparse_tuning_result_v[i])
      Xu_i <- t(splitted_data[[i]]) %*% as.matrix(u_new)
      v_part_raw <- sparse_pen_fun(y = Xu_i,
```

```
                                  tuning_parameter = sparse_param,
                                  type = sparse_tuning_type)
      if (all(v_part_raw == 0)) v_part_raw <- Xu_i

      if (conditional && !is.null(Omega_u)) {
        norm_u2 <- as.numeric(norm_vec(u_new)^2)
        R_u <- as.numeric(t(u_new) %*% Omega_u %*% u_new / norm_u2)
        v_part <- S_alpha_v[[i]] %*% v_part_raw / (1 + R_u)
      } else {
        v_part <- S_alpha_v[[i]] %*% v_part_raw
      }
      v_new <- c(v_new, v_part)
    }

    v_new <- v_new / norm_vec(v_new)
    errors <- sum((v_new - v_old)^2)
    v_old <- v_new
  }
  u_new <- u_new / norm_vec(u_new)
  return(list(v_new, u_new))
}

opt_alpha_v <- function(X,
                        n_var,
                        ncol,
                        S_alphas_v,   # List of length = n_iter, each element is list of length = n_var
                        S_alphas_u,   # Matrix (n × n)
                        alphas,       # Matrix of alpha_v values (n_iter × n_var)
                        alpha_u,      # scalar
                        sparse_tuning_result_u,
                        sparse_tuning_result_v,
                        Omega_u,
                        sparse_tuning_type,
                        conditional = TRUE) {  #  argument

  alphas <- data.frame(alphas)
  n_iter <- nrow(alphas)
  n <- nrow(X)
  m <- ncol(X)
  GCV <- numeric(n_iter)

  if (all(alphas == 0)) {
    return(list(GCV_v = Inf,
                opt.alpha_v = rep(0, n_var),
                opt_s.alpha_v = replicate(n_var, diag(m / n_var), simplify = FALSE),
                GCVdf_v = data.frame(alphas, rep(Inf, n_iter))))
  } else {
    for (i in 1:n_iter) {
      S_alpha_v <- S_alphas_v[[i]]
      alpha <- alphas[i, ]

      power_result <- power_algo2(data = X,
                                  n_var = n_var,
```

```
                                ncol = ncol,
                                sparse_tuning_result_u = sparse_tuning_result_u,
                                sparse_tuning_result_v = sparse_tuning_result_v,
                                S_alpha_v = S_alpha_v,
                                S_alpha_u = S_alphas_u,
                                sparse_tuning_type = sparse_tuning_type,
                                conditional = conditional,
                                Omega_u = if (conditional) Omega_u else NULL,
                                Omega_v = NULL)

      v <- power_result[[1]]
      u <- power_result[[2]]

      norm_u2 <- as.numeric(norm_vec(u)^2)
      Xu_proj <- as.vector(t(X) %*% u / norm_u2)
      R_u <- as.numeric(t(u) %*% Omega_u %*% u / norm_u2)

      numer <- (1 / m) * norm_vec(Xu_proj - v)^2
      trace_total <- sum(sapply(S_alpha_v, function(Sk) sum(diag(Sk))))
      denom <- (1 - (1 / m) * (trace_total / (1 + alpha_u * R_u)))^2

      GCV[i] <- numer / denom
    }

    opt.alpha <- alphas[which.min(GCV), ]
    opt_s.alpha <- S_alphas_v[[which.min(GCV)]]

    return(list(GCV_v = GCV,
                opt.alpha_v = opt.alpha,
                opt_s.alpha_v = opt_s.alpha,
                GCVdf_v = data.frame(alphas, GCV)))
  }
}


GCV_result_v = opt_alpha_v(X = X_temp,
                           n_var = n_var,
                           ncol = ncol,
                           S_alphas_v = S_alpha_list_v, # A list (length = n_var)
                           S_alphas_u = diag(nrow(X_temp)),
                           alphas = setNames(data.frame(unique(smooth_tuning_col[,1]) ,
                                                        rep(0, length(unique(smooth_tuning_col[,1])))),
                           alpha_u = 1,
                           sparse_tuning_result_u = 0,
                           sparse_tuning_result_v = c(0,0),
                           Omega_u = diag(nrow(X_temp)),
                           sparse_tuning_type = sparse_tuning_type,
                           conditional = TRUE)


GCV_result_v$opt.alpha_v
```

```
##    alpha1 alpha2
## 10     32      0
```

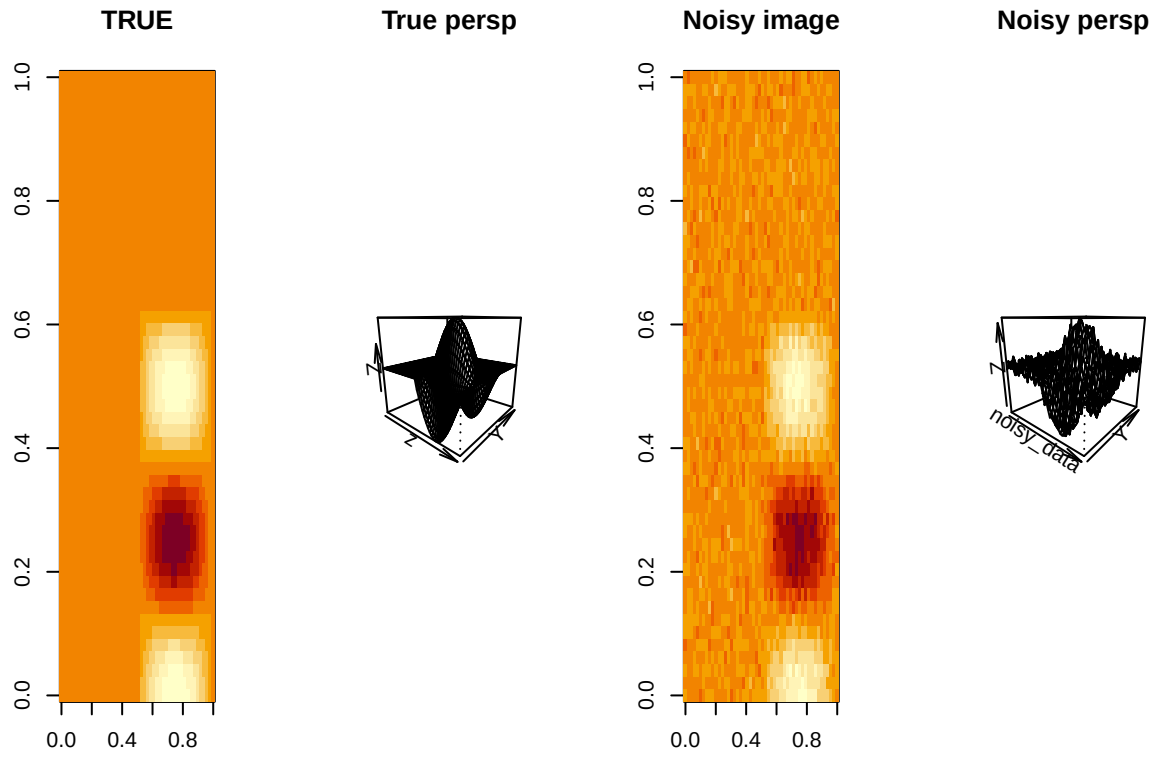
```
GCV_result_v$GCVdf_v
```

```
##          alpha1 alpha2       GCV
## 1  9.313226e-10      0 164.77628
## 2  1.379661e-08      0 161.42129
## 3  2.043829e-07      0 136.65774
## 4  3.027727e-06      0 102.74529
## 5  4.485274e-05      0  87.26454
## 6  6.644482e-04      0  80.57441
## 7  9.843133e-03      0  77.47087
## 8  1.458161e-01      0  75.99963
## 9  2.160119e+00      0  75.52028
## 10 3.200000e+01      0  75.47074
```

4.1. Sparsity on u (1-SE)

To test Sparsity and Smoothness on u, we need a set of data that has functional behavior on both sides to be able to incorporating regularization on both the left and right singular vectors in the Singular Value Decomposition (SVD) of the data matrix. As a result, we can work with the following data:

```
x <- seq(0,2*pi, len = 50)
y1 <- sin(x); y1[1:26] <- 0
y2 <- cos(2*x); y2[32:50] <- 0
z<- outer(y1,y2)

par(mfrow = c(1,4))
image(z, main = "TRUE")
persp(z, theta = 40, main = "True persp")

set.seed(123)
noisy_data <- z + rnorm(length(z), sd = 0.1)
image(noisy_data, main = "Noisy image")
persp(noisy_data, theta = 40, main = "Noisy persp")
```



Based on the images shown above, level of sparsity for u needs to be 25. Now, we can implement sparsity on u:

```
sparse_tuning_u <- attr(object_list, "Sparsity_parameter")

S_alpha_v <- list()
for (i in 1:n_var) {
  S_alpha_v[[i]] <- diag(ncol[,i])
}

CV_score_sparse_u <- CV_score_sparse_v <- GCV_score_smooth_u <- GCV_score_smooth_v <- Inf
CV_scores_result_u <- CV_scores_result_v <- c()
result = c()

# Step 1: Sparsity on u  with no smoothness
for (sparse_tuning_single in sparse_tuning_u) {
  sparse_score = cv_sparse_row(data=X_temp,
                               ncol = ncol,
                               n_var = n_var,
                               S_alpha_v = S_alpha_v,
                               S_alpha_u = diag(nrow(X_temp)),
                               K_fold = K_fold,
                               sparse_tuning_result_u = sparse_tuning_single,
                               sparse_tuning_result_v = rep(0, n_var),
                               sparse_tuning_type = sparse_tuning_type)

  CV_scores_result_u <- c(CV_scores_result_u, sparse_score)
```

```
  if (sparse_score <= CV_score_sparse_u) {
    CV_score_sparse_u = sparse_score
    gamma_u = sparse_tuning_single
  }
}
gamma_u
```

```
## [1] 0
```

```
plot(CV_scores_result_u, type = 'l')
```